
Estrategia de prueba

Qué se automatiza, qué no, y dónde se prueba realmente

Documento **SLCP-DOC-011** – Versión **1.0**

6 de agosto de 2026

Este documento responde qué pruebas pueden automatizarse además de las de aceptación ya especificadas, y qué pruebas siguen siendo necesarias aunque no se automaticen. Antes de enumerarlas plantea la cuestión que condiciona toda la estrategia y que no aparece en los tratados generales: en una plataforma que genera aplicaciones, probar cada salida generada es casi siempre el sitio equivocado donde poner el esfuerzo. Lo que puede fallar es el generador, y ahí es donde deben concentrarse las pruebas.

Índice

1	Precisión de partida	2
1.1	Qué es y qué no es TDD aquí	2
1.2	Verificación y validación siguen siendo cosas distintas	2
2	La cuestión que condiciona la estrategia	2
2.1	¿Probar la salida o el generador?	2
3	Pruebas automatizables	3
3.1	Las cuatro que merecen comentario aparte	4
4	Pruebas necesarias que no se automatizan	4
4.1	El riesgo específico de esta plataforma	5
5	Requisitos	5
6	Reparto del esfuerzo	7
7	Asuntos abiertos	7

1 Precisión de partida

1.1 Qué es y qué no es TDD aquí

Conviene distinguir tres prácticas que suelen mezclarse bajo el rótulo de escribir las pruebas primero.

Práctica	Nivel	Estado en el proyecto
Desarrollo dirigido por pruebas	Unidad de código; es una técnica de diseño, no de verificación	Aplicable al núcleo escrito a mano
Desarrollo dirigido por pruebas de aceptación	Requisito; fija qué significa estar satisfecho	Ya especificado en SPC-02 y SPC-04
Oráculo antes que implementación	Artefacto generado; condición de promoción	Ya establecido en TGT-08

La disciplina de escribir la prueba primero ya está institucionalizada en el proyecto por TGT-08, y no como buena práctica sino como requisito con criterio de verificación. El incremento 0.1 la aplicó de hecho: el oráculo se selló antes de escribir el módulo, y encontró un defecto que la implementación por sí sola habría ocultado.

1.2 Verificación y validación siguen siendo cosas distintas

TRC-13 ya lo estableció y aquí tiene consecuencia práctica: todo lo automatizable pertenece a la verificación, que responde si el producto se construyó conforme a la especificación. La validación, que responde si la especificación era la correcta, no se automatiza en ningún grado. Esa es la razón de fondo por la que la segunda parte de este documento existe.

2 La cuestión que condiciona la estrategia

2.1 ¿Probar la salida o el generador?

En un proyecto convencional, cada unidad de código se escribe una vez y se prueba una vez. En esta plataforma, un repositorio generado a partir de una plantilla determinista es un ejemplar entre muchos: si la plantilla es correcta, todos sus ejemplares lo son; si es incorrecta, todos fallan del mismo modo.

Generar pruebas unitarias para cada repositorio generado produce, en consecuencia, un volumen enorme de pruebas que comprueban una y otra vez la misma propiedad de la misma plantilla. Se paga el coste de ejecutarlas y de mantenerlas sin obtener información nueva.

Regla que gobierna el reparto del esfuerzo. En las capas deterministas se prueba el generador, no cada salida. En la capa interpretativa se prueba cada artefacto, porque cada uno es distinto y ninguno se deriva por regla.

Tabla 1: Dónde se prueba cada capa

Capa	Qué puede fallar	Objeto de prueba	Técnica
L0-L2	La plantilla, el descriptor de destino o la transformación de nomenclatura	El generador	Pruebas de aprobación sobre proyectos de referencia y validación de esquema
L3	La traducción de la regla declarativa a código	El generador y la regla	Cobertura de todas las reglas y transiciones, derivada de la propia declaración
L4	La implementación concreta de cada artefacto	Cada artefacto	Oráculo sellado, propiedades y mutación

Obsérvese que en L3 la prueba se deriva de la especificación declarativa y por tanto es completa por construcción respecto de ella: cubrir todas las filas de una tabla de decisión no requiere ingenio, solo recorrerla. Esa es la razón por la que conviene empujar tanta lógica como sea posible hacia L3, como ya señaló SLCP-ADR-0002.

3 Pruebas automatizables

Se organizan por lo que verifican y no por la herramienta que las ejecuta. La columna de nivel indica si aplican a la plataforma, al proyecto gestionado o a la aplicación generada.

Tabla 2: Catálogo de pruebas automatizables

Prueba	Qué verifica	Nivel	Cuándo se escribe
Unitaria	Una unidad de código aislada de sus colaboradores	N1, N3 en L4	Antes de la implementación
Integración	La colaboración real con base de datos, colas y servicios	N1, N3	Con el contrato, antes del código
De contrato	Que proveedor y consumidor respetan la descripción OpenAPI	N1, N3	Al fijar el contrato
De sistema	Los escenarios ejecutados contra la aplicación desplegada	N3	Ya existen: son los de SPC-02
De aprobación del generador	Que un proyecto de referencia se genera igual que la salida aceptada	N1	Al crear cada plantilla
De determinismo	Que dos generaciones con la misma entrada producen la misma salida	N1	Junto al generador
De regla declarativa	Que se cubren todas las filas de la tabla y todas las transiciones del estado	N3 en L3	Derivada de la declaración
De propiedades	Invariantes que deben cumplirse para toda entrada, no solo para las enumeradas	N1, N3	Con el requisito
De mutación	La fuerza del oráculo: si el oráculo detecta defectos introducidos a propósito	N1, N3	Tras el oráculo

Prueba	Qué verifica	Nivel	Cuándo se escribe
Metamórfica	Relaciones entre entradas y salidas cuando no existe oráculo exacto	N1	Con el analizador asistido
De conformidad arquitectónica	Que el código respeta la estructura declarada en la vista de desarrollo	N1, N3	Con la vista
Estática y de composición	Debilidades de Int [1], dependencias vulnerables, secretos filtrados	N1, N3	Continua
De rendimiento y carga	Los atributos de eficiencia de Int [2]	N1, N3	Con el atributo de calidad
De accesibilidad automatizable	La porción comprobable por máquina de Wor [3]	N1, N3	Con la interfaz
De migración	Que cada migración aplica, revierte y es idempotente	N1, N3	Con la migración
De ida y vuelta	Que exportar e importar no pierde información (INT-09)	N1	Con el formato
De propagación	Transaccionalidad, convergencia y ausencia de ciclos (PRO-03, PRO-08)	N1	Con el motor
Negativa de auditoría	Que no existe ruta capaz de borrar un evento (TRC-24)	N1	Con el almacén

3.1 Las cuatro que merecen comentario aparte

Mutación. Es la única técnica automatizada capaz de responder si el oráculo sirve para algo. Introduce defectos artificiales en el código y comprueba si las pruebas los detectan [4]. Su valor aquí es directo: SLCP-ADR-0002 identificó como riesgo residual R-01 la debilidad del oráculo, y advirtió que unas pruebas en verde solo demuestran ausencia de los fallos que ellas exploran. La mutación es la mitigación de ese riesgo, y probablemente la aportación más valiosa de todo este catálogo.

Metamórficas. El analizador asistido de requisitos no tiene salida correcta única: no cabe escribir la lista exacta de hallazgos que debe producir. Lo que sí cabe es fijar relaciones que deben cumplirse entre entradas y salidas, por ejemplo que añadir ambigüedad a un enunciado no puede reducir el número de hallazgos [5]. Es el enfoque que recomienda Int [6] para sistemas basados en IA y el único aplicable a ANA-01.

Conformidad arquitectónica. Comprueba de forma automática que el código respeta la estructura de módulos y dependencias declarada. Convierte ARQ-08 en algo verificable en cada integración en lugar de en una intención documentada, que es la diferencia entre una arquitectura viva y un diagrama antiguo.

Determinismo. TRC-23 obliga a declarar el régimen de determinismo de cada artefacto. Esa declaración debe probarse, no solo escribirse: generar dos veces con la misma entrada y comparar. Sin esta prueba, la afirmación de reproducibilidad es una promesa sin respaldo.

4 Pruebas necesarias que no se automatizan

Ninguna de las siguientes puede sustituirse por herramienta, y todas son necesarias. El motivo común es que responden a preguntas de juicio y no de cálculo.

Tabla 3: Pruebas y revisiones no automatizables

Actividad	Qué responde	Por qué no se automatiza
Revisión del oráculo	Si los escenarios cubren lo que importa	Una suite verde solo prueba lo que explora; lo omitido es invisible para ella
Validación con la parte interesada	Si la especificación era la correcta	La corrección de la especificación no se deduce de la especificación
Pruebas exploratorias	Qué falla que nadie previó	Toda prueba escrita busca lo que su autor imaginó
Usabilidad	Si la persona logra su objetivo sin fricción	No hay definición mecánica de fricción
Accesibilidad de juicio	Si el texto alternativo dice algo útil y el orden de foco tiene sentido	La herramienta detecta la ausencia del atributo, no la calidad de su contenido
Modelado de amenazas y prueba de intrusión	Qué haría alguien con intención	El atacante no sigue el guión que se le escriba
Revisión muestreada del código generado	Si lo generado es comprensible y mantenible	La legibilidad no es una medida estática
Revisión de protección de datos	Si el tratamiento se ajusta a su finalidad	Es un juicio jurídico
Ensayo de recuperación	Si el procedimiento funciona cuando hace falta	Su valor está en ejecutarlo con personas reales bajo presión

4.1 El riesgo específico de esta plataforma

Una herramienta que genera código y genera sus pruebas corre un riesgo particular: que ambos deriven del mismo malentendido. Si el requisito se interpretó mal, el código hará lo equivocado y la prueba comprobará que lo hace correctamente. Las dos estarán de acuerdo y las dos estarán mal.

Ninguna técnica automatizada detecta esa situación, porque no hay contradicción interna que detectar. Solo la detecta alguien que lea el escenario y lo contraste con lo que necesitaba. Esa es la razón por la que SPC-05 exige que el propietario del producto apruebe los escenarios, y por la que la revisión del oráculo encabeza la tabla anterior.

5 Requisitos

VER-01 Prueba antes que implementación en el núcleo. [N1]

El código escrito a mano de la plataforma debe desarrollarse escribiendo primero la prueba que lo verifica.

Verificación: Ningún módulo del núcleo se incorpora sin pruebas anteriores a su implementación, comprobable en el histórico del repositorio.

VER-02 Prueba del generador, no de cada salida. [N1]

En las capas deterministas la verificación debe recaer sobre el generador mediante pruebas de aprobación sobre proyectos de referencia, y no debe generarse una batería de pruebas unitarias redundantes por cada proyecto producido.

Verificación: Un cambio en una plantilla hace fallar la prueba de aprobación correspondiente sin necesidad de generar ningún proyecto nuevo.

VER-03 Prueba de determinismo. [N1][N3]

La plataforma debe verificar que dos generaciones con la misma entrada producen salidas idénticas en las capas declaradas deterministas conforme a TRC-23.

Verificación: La prueba falla ante cualquier fuente de no determinismo no declarada, incluidas marcas de tiempo y ordenaciones inestables.

VER-04 Prueba de mutación sobre el oráculo. [N1][N3]

La fuerza del oráculo debe medirse mediante prueba de mutación, con un umbral mínimo configurable por proyecto. Un oráculo que no alcance el umbral debe reportarse como débil.

Verificación: Ningún artefacto de lógica de negocio se promueve con un oráculo por debajo del umbral sin decisión humana registrada.

VER-05 Prueba metamórfica del análisis asistido. [N1]

El analizador de requisitos debe verificarse mediante relaciones metamórficas declaradas, dado que carece de salida correcta única.

Verificación: Existe un conjunto de relaciones declaradas y su incumplimiento se reporta como defecto del analizador.

VER-06 Conformidad arquitectónica automatizada. [N1][N3]

La estructura de módulos y dependencias declarada en la vista de desarrollo debe verificarse de forma automática en cada integración.

Verificación: Una dependencia que viole la estructura declarada detiene la integración.

VER-07 Integración contra motor real. [N1][N3]

Las pruebas de integración deben ejecutarse contra un motor de base de datos real en contenedor efímero, y no contra sustitutos en memoria.

Verificación: Las diferencias de dialecto entre destinos se detectan en integración y no en producción.

VER-08 Cadena estática obligatoria. [N1][N3]

Toda integración debe ejecutar análisis estático conforme a Int [1], análisis de composición de dependencias y búsqueda de secretos.

Verificación: El fallo de cualquiera de los tres detiene la integración.

VER-09 Revisión independiente del oráculo. [N1][N2]

El oráculo de un artefacto debe revisarlo una persona distinta de quien lo generó o lo redactó,

conforme al principio de ROL-04.

Verificación: No existe oráculo aprobado por su propio autor.

VER-10 Pruebas no automatizables planificadas. [N1][N2]

Las actividades de la Tabla 3 deben modelarse como elementos de la estructura de desglose, con sus recursos y su avance.

Verificación: Lo no automatizable figura en el plan y se mide igual que el resto del trabajo.

VER-10 no es una formalidad. Lo que no se planifica no se hace, y las pruebas manuales son las primeras que desaparecen cuando aprieta el calendario, precisamente porque nada falla en rojo cuando se omiten. Situarlas en la estructura de desglose las hace visibles y les da oráculo, recursos y responsable como a cualquier otra tarea.

VER-11 Sesiones exploratorias con registro. [N1][N3]

Las pruebas exploratorias deben realizarse por sesiones con objetivo declarado, duración acotada y registro de hallazgos, y sus hallazgos deben incorporarse como defectos trazados conforme a TRC-18.

Verificación: Cada sesión deja constancia de qué se exploró y qué se encontró.

VER-12 Documentación conforme a la norma de prueba. [N1][N2]

Los productos de trabajo de prueba deben ajustarse a las plantillas de Int [7] y las técnicas de diseño a Int [8].

Verificación: Cada conjunto de pruebas declara la técnica de diseño empleada.

6 Reparto del esfuerzo

La distribución habitual en forma de pirámide, con muchas pruebas unitarias en la base, se deforma en esta plataforma porque el riesgo no está repartido de manera uniforme.

Dónde está el riesgo	Esfuerzo	Motivo
El generador y sus plantillas	Alto	Un defecto se replica en todos los proyectos
La lógica de nivel L4	Alto	Cada artefacto es distinto y ninguno se deriva por regla
La integración y los dialectos	Alto	Es donde fallan las diferencias entre destinos
El motor de propagación	Alto	Un fallo corrompe la coherencia de todo el proyecto
El CRUD generado en L0–L2	Bajo	Correcto por construcción si la plantilla lo es

7 Asuntos abiertos

Tabla 4: Preguntas abiertas de este documento

ID	Pregunta	Valor por defecto propuesto
Q-47	Qué umbral de mutación se exige	Sin umbral bloqueante en la primera iteración; se mide y se calibra con datos del caso semilla
Q-48	Se generan pruebas unitarias para el código de L0 a L2	No, conforme a VER-02; se prueban las plantillas
Q-49	Quién ejecuta las pruebas exploratorias	Miembros del equipo distintos de quien construyó la parte explorada
Q-50	Se exige prueba de rendimiento desde la primera liberación	Solo si existe un atributo de calidad de eficiencia declarado y significativo
Q-51	Se prueba la accesibilidad de la aplicación generada o solo de la plataforma	De ambas; la generada hereda la comprobación automática de sus plantillas

Referencias

- [1] *ISO/IEC 5055:2021 Information technology — Software measurement — Software quality measurement — Automated source code quality measures*. International Organization for Standardization, Geneva, Switzerland, 2021. <https://www.iso.org/standard/80623.html>.
- [2] *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. International Organization for Standardization, Geneva, Switzerland, 2023. Edición 2, 2023-11; cancela y reemplaza a ISO/IEC 25010:2011 junto con ISO/IEC 25002 e ISO/IEC 25019. <https://www.iso.org/standard/78176.html>.
- [3] *Web Content Accessibility Guidelines (WCAG) 2.2*. World Wide Web Consortium (W3C), 2023. W3C Recommendation. <https://www.w3.org/TR/WCAG22/>.
- [4] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. ISSN 0098-5589. doi: 10.1109/TSE.2010.62.
- [5] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):1–27, 2018. ISSN 0360-0300. doi: 10.1145/3143561.
- [6] *ISO/IEC TR 29119-11:2020 Software and systems engineering — Software testing — Part 11: Guidelines on the testing of AI-based systems*. International Organization for Standardization, Geneva, Switzerland, 2020. Informe técnico, no norma. <https://www.iso.org/standard/79016.html>.
- [7] *ISO/IEC/IEEE 29119-3:2021 Software and systems engineering — Software testing — Part 3: Test documentation*. International Organization for Standardization, Geneva, Switzerland, 2021. <https://www.iso.org/standard/79429.html>.
- [8] *ISO/IEC/IEEE 29119-4:2021 Software and systems engineering — Software testing — Part 4: Test techniques*. International Organization for Standardization, Geneva, Switzerland, 2021. Edición 2; cancela y reemplaza a ISO/IEC/IEEE 29119-4:2015. <https://www.iso.org/standard/79430.html>.